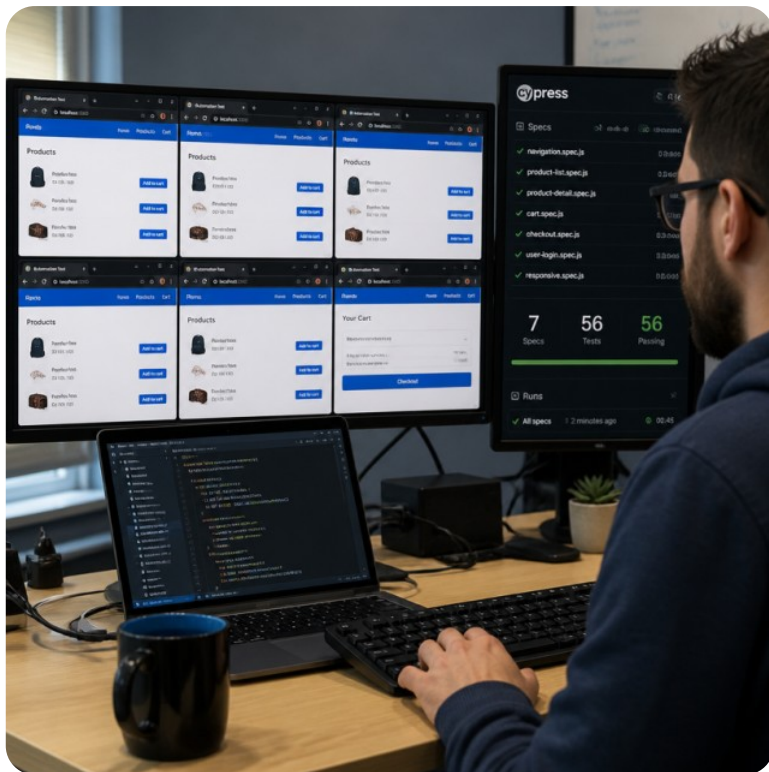


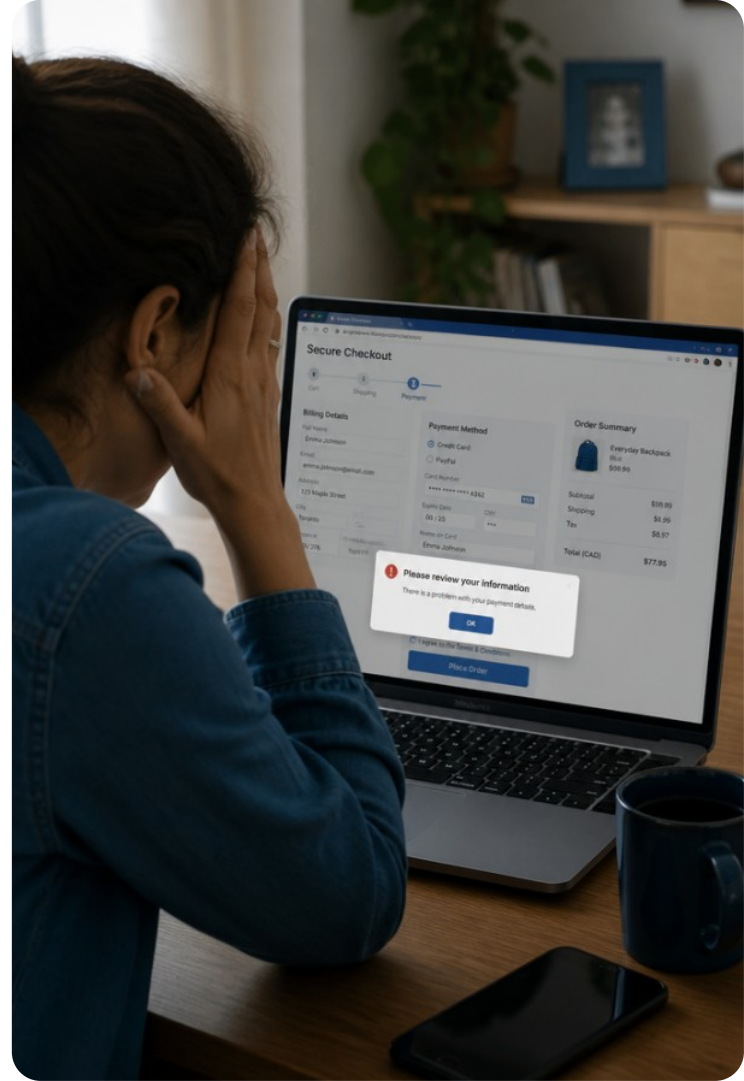


Testes End-to-End com Cypress e Playwright

Automação de jornadas completas no navegador para aplicações web modernas.

Um Botão Invisível Quebrou as Vendas?

Todas as APIs responderam com status 200 e os testes unitários passaram. Mesmo assim, nenhum cliente conseguiu finalizar a compra porque um banner sobrepôs o botão de pagamento.



Objetivos da Aula

Neste encontro do Módulo Específico de Testes de Frontend, focaremos na camada do topo da pirâmide:

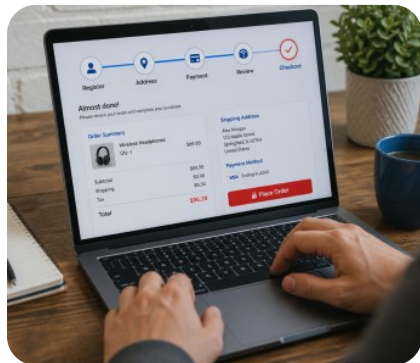
Diferenciar testes unitários e de integração dos testes End-to-End.

Automatizar fluxos reais de navegação e interação com Cypress e Playwright.

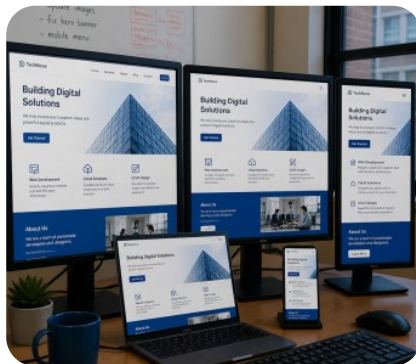
Identificar e mitigar intermitências (*flakiness*) em execuções de testes.



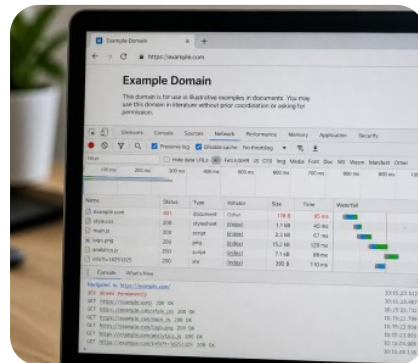
Vocabulário Essencial de E2E



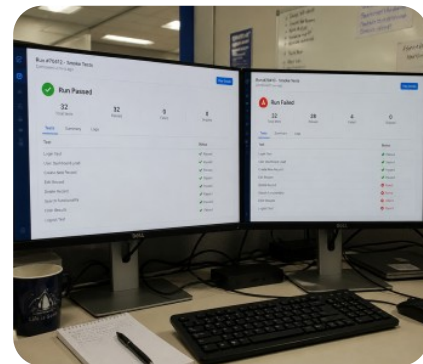
Jornada - Passos que o usuário percorre para cumprir uma meta real no sistema.



Navegador - Cliente (Chromium, Firefox, WebKit) que executa HTML, CSS e JavaScript.

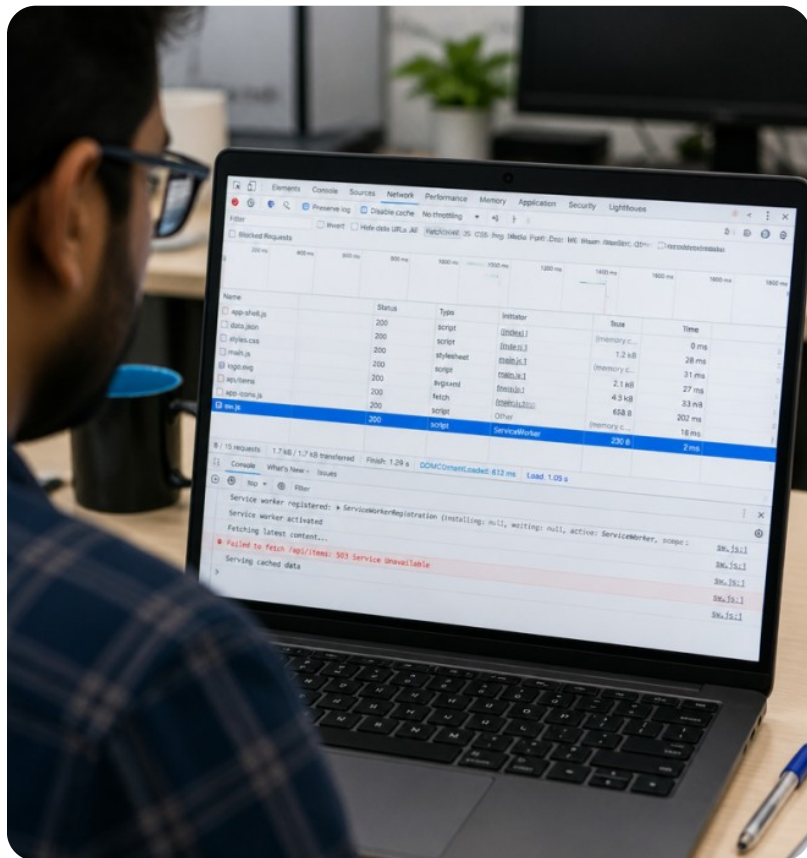


Navegação - Transição entre rotas e URLs, disparando requisições e renderizações.



Intermitência - Instabilidade (*flakiness*) onde o teste falha sem mudança no código.

Recapitulação: Integração e MSW



REVISÃO

Revisamos testes de componentes e serviços assíncronos:

MSW: Intercepta requisições HTTP via rede sem alterar o código.

Assincronismo: Usamos para aguardar promessas e atualizações no DOM.



Ponto-chave

Testes de integração validam comunicação entre componentes em ambiente simulado (jsdom), sem navegador real.

Quiz: Interceptação e Assincronismo



Pergunta 1:

Qual é a principal vantagem de utilizar o Mock Service Worker (MSW) em testes de integração de frontend?

Pergunta 2:

Por que usamos consultas assíncronas como `findByRole` em vez de `getByRole` ao lidar com respostas de rede?

Pergunta 3:

O que ocorre quando um teste de integração não espera a resolução de uma promessa antes de avaliar asserções?

Quiz: Interceptação e Assincronismo



Resposta 1:

Interceptar requisições HTTP reais via Service Worker sem alterar o código dos serviços ou componentes.

Resposta 2:

Porque findByRole aguarda o elemento aparecer no DOM respeitando um tempo limite padrão, tratando o assincronismo.

Resposta 3:

O teste avalia o DOM prematuramente no estado de carregamento inicial, gerando falhas falsas-negativas.

O que são Testes End-to-End?

CONCEITO

Testes **End-to-End (E2E)** validam o sistema completo em um **navegador real**, simulando interações humanas de mouse e teclado.

- Renderiza CSS e layout reais.
- Testa fluxos com APIs, cookies e sessões reais.

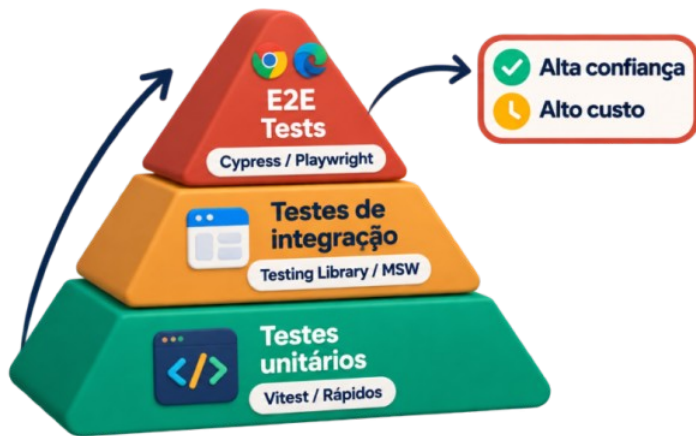


Lembre-se

E2E foca na jornada do usuário e no fluxo de valor, não em funções isoladas.

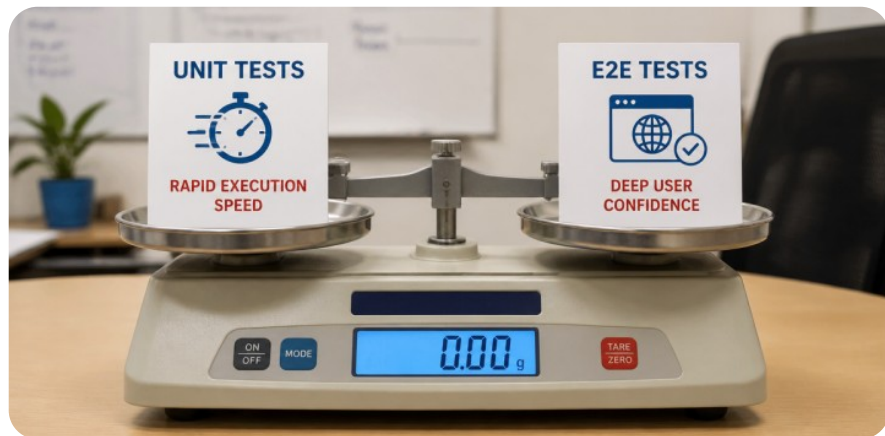


Pirâmide de Testes: Posição do E2E



O Topo da Pirâmide

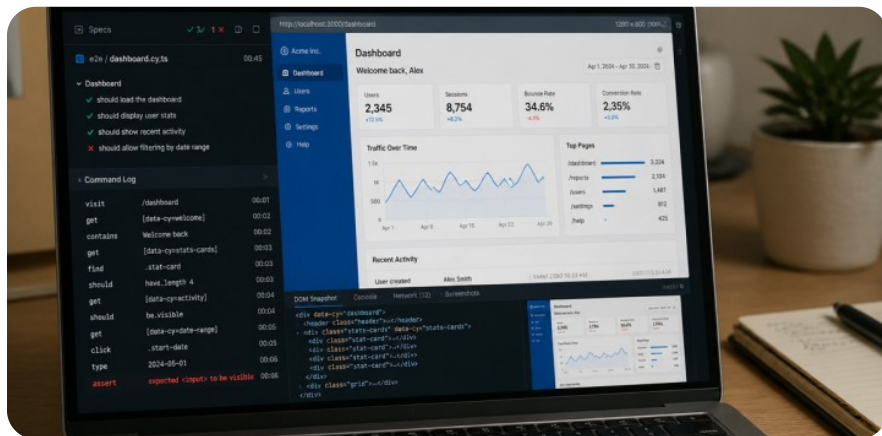
Os testes E2E ficam no topo: oferecem a maior **confiança de negócio**, mas possuem execução mais lenta e maior custo de manutenção.



Proporção Recomendada

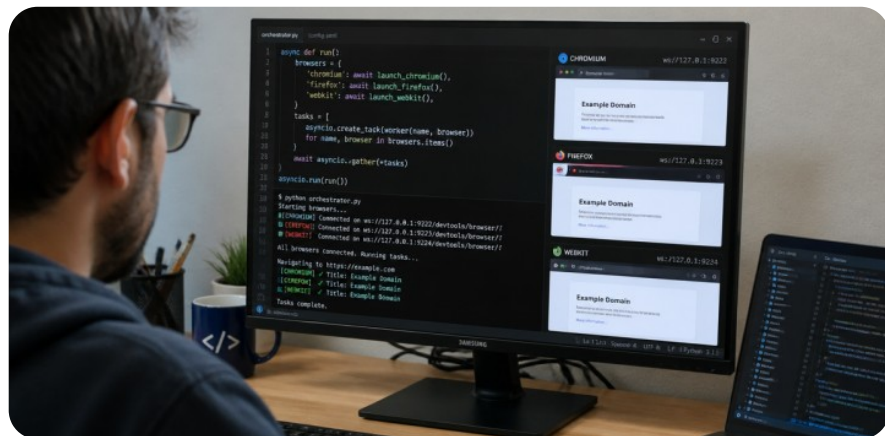
Siga a proporção saudável: cerca de **10% da suite** deve ser composta por fluxos E2E críticos, evitando testes redundantes que deixam a esteira lenta.

Ecossistema: Cypress vs Playwright



Cypress

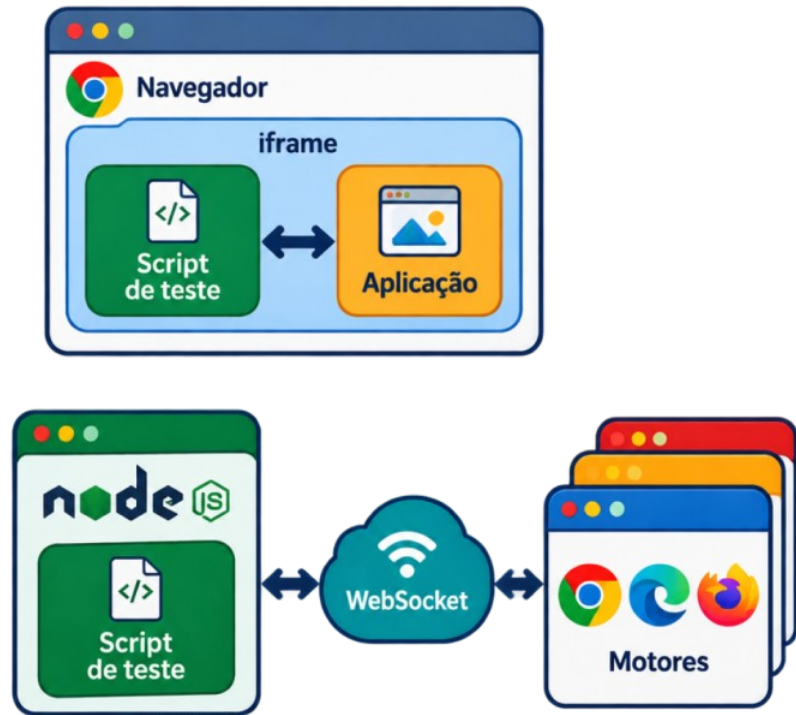
Executa no mesmo loop de eventos do navegador. Oferece depuração visual imediata e excelente experiência de desenvolvimento para aplicações web.



Playwright

Criado pela Microsoft, usa protocolos nativos (CDP). Suporta testes multipágina, multi-abas e navegadores Chromium, Firefox e WebKit com alta velocidade.

Arquitetura e Execução Interna



ARQUITETURA

Entender a comunicação ferramenta-app evita erros:

Cypress: roda como script em iframe, controlando eventos em tempo real.

Playwright: teste roda em processo Node.js, instruindo navegador via WebSockets no motor de renderização.



Ponto-chave

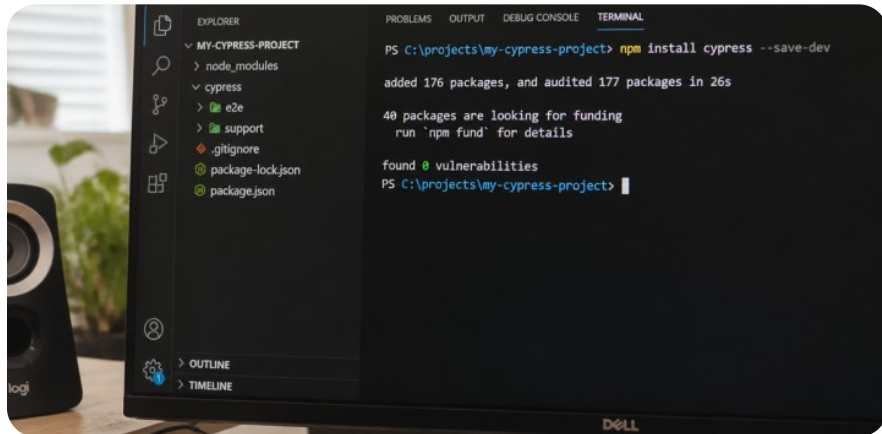
Playwright gerencia múltiplas abas/domínios; Cypress foca em isolamento e simplicidade.

Vídeo: E2E na Prática Profissional

Pipelines E2E garantem deploys contínuos e seguros.

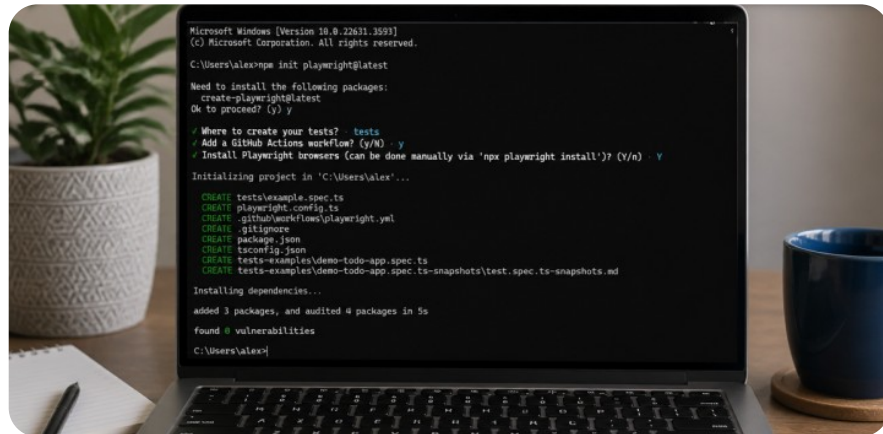


Instalação e Estrutura de Projeto



Estrutura do Cypress

Instalado com `npm install cypress --save-dev`. Cria a pasta `cypress` para especificações e a pasta `e2e` para comandos customizados.



Estrutura do Playwright

Inicializado com `npm init playwright@latest`. Gera a pasta `tests` e a configuração pré-fabricada de navegadores e integração contínua.

Navegação: O Comando `cy.visit()`

COMANDOS

Toda jornada no Cypress inicia com a URL inicial:

- : acessa a rota base da configuração.

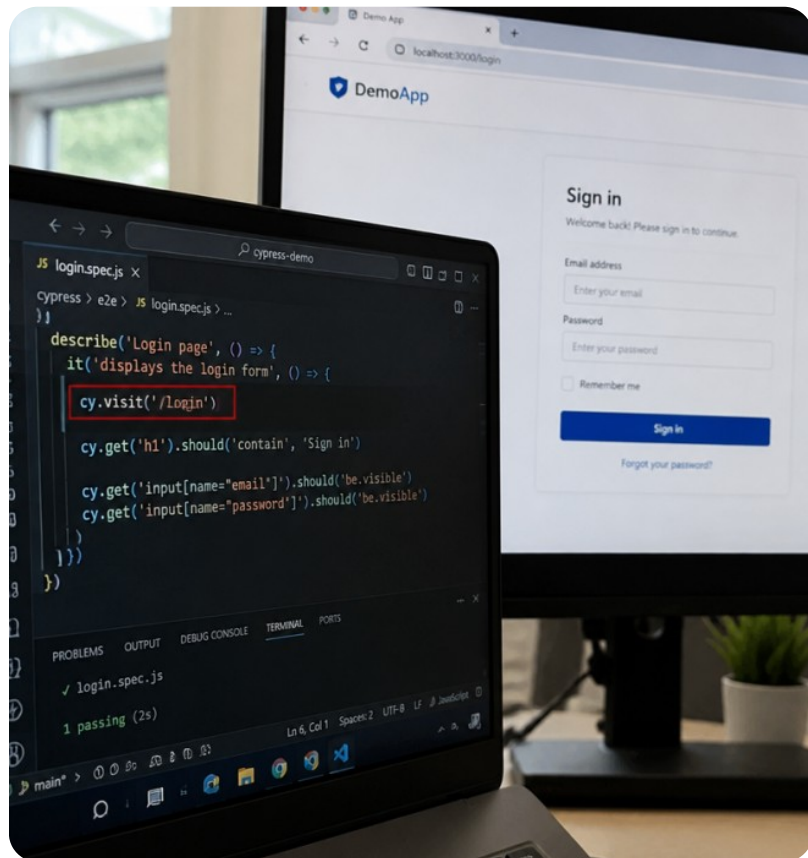
Aguarda automaticamente eventos e .

- Falha se o servidor retornar erro 4xx ou 5xx.

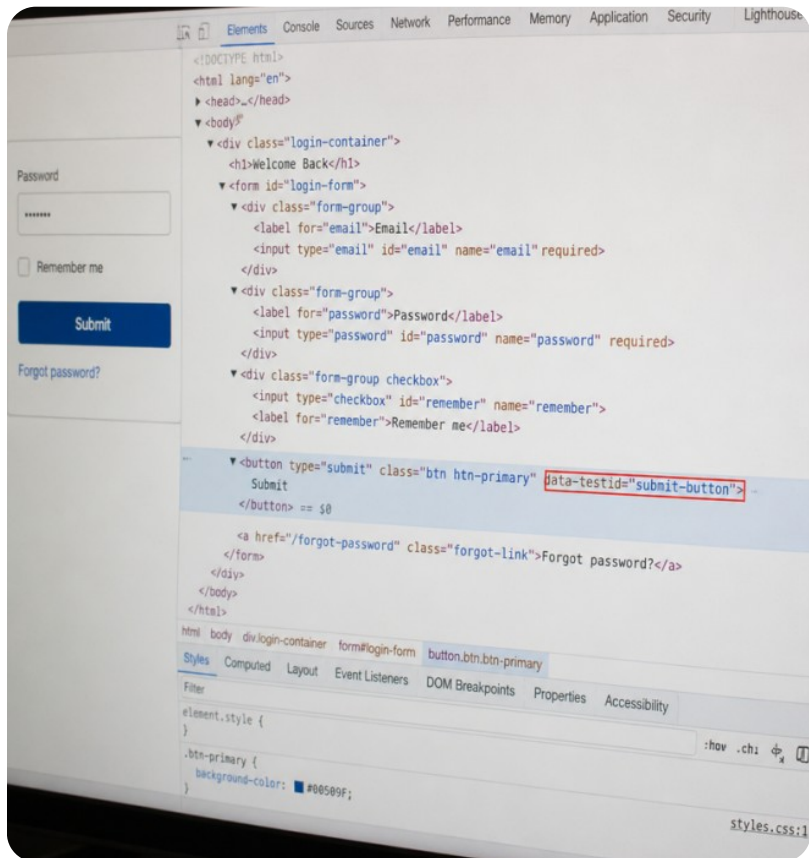


Exemplo

Defina `baseUrl` no `cypress.config.js` para evitar fixar o endereço em cada teste.



Seleção de Elementos no Cypress



SELETORES

Cypress localiza elementos no DOM:

- : busca via seletores CSS.
- : busca por texto visível.

Ideal: use atributos em vez de classes CSS.



Atenção

Evite classes de estilo: mudanças visuais quebram testes.

Ações Automatizadas do Usuário



Digitação

`cy.get('#email').type('dev@senai.br')`
simula digitação
tecla por tecla.

Seleção

`cy.get('#estado').select('SP')` escolhe
opções em menus
suspensos.

Clique

`cy.get('[data-testid="btn-submit"]').click()`
dispara evento de
clique.

Submissão

`cy.get('form').submit()` envia
formulários
simulando o Enter.

Assertivas Encadeadas e should()

ASSERÇÕES

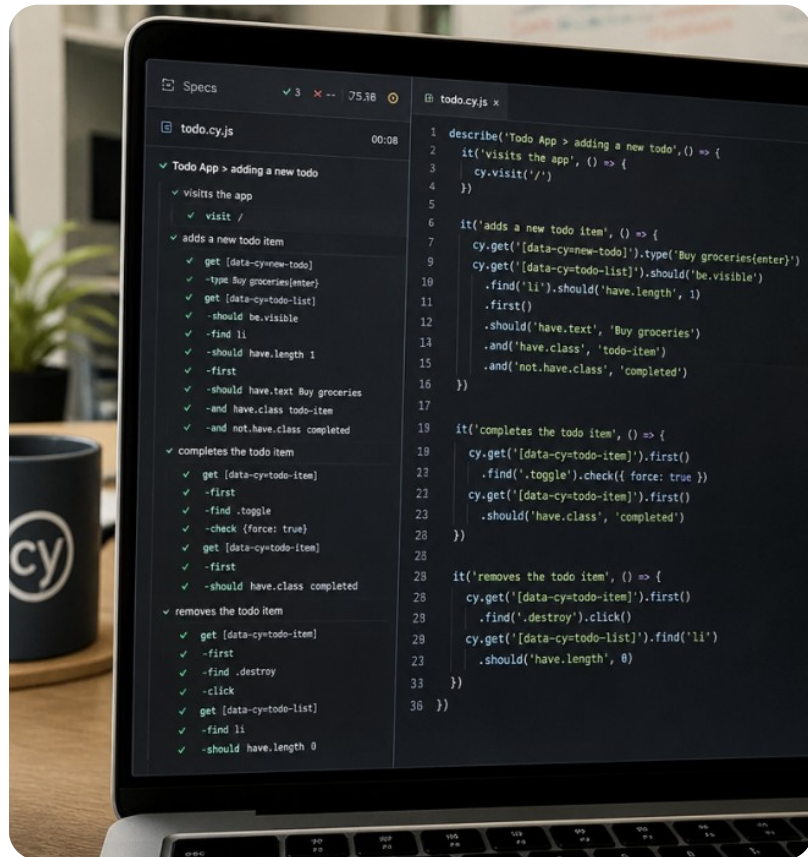
No Cypress, as validações usam sintaxe BDD baseada em Chai através do comando :

Encadeamento: .



Ponto-chave

O comando `should()` executa polling automático: ele repete a verificação até passar ou atingir o tempo limite (timeout).



Verificação: Seletores Confiáveis

Qual é a melhor estratégia de seleção de elementos para garantir testes E2E resistentes a refatorações visuais?

1. Selecionar pela ordem posicional das tags HTML, como `div > p:nth-child(2)`.
2. Utilizar classes CSS de layout e cores, como `.btn-primary-blue`.
3. Utilizar atributos dedicados de teste, como `[data-testid='btn-salvar']`.
4. Copiar o caminho XPath absoluto a partir da raiz `html/body/div`.

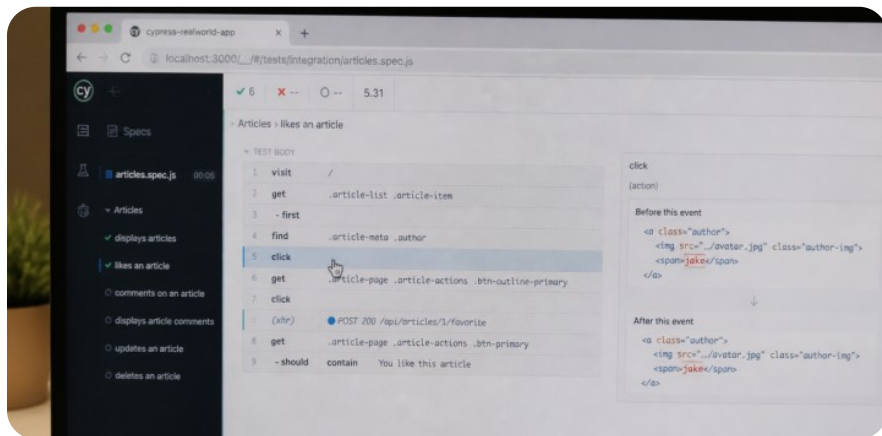
Verificação: Seletores Confiáveis



Qual é a melhor estratégia de seleção de elementos para garantir testes E2E resistentes a refatorações visuais?

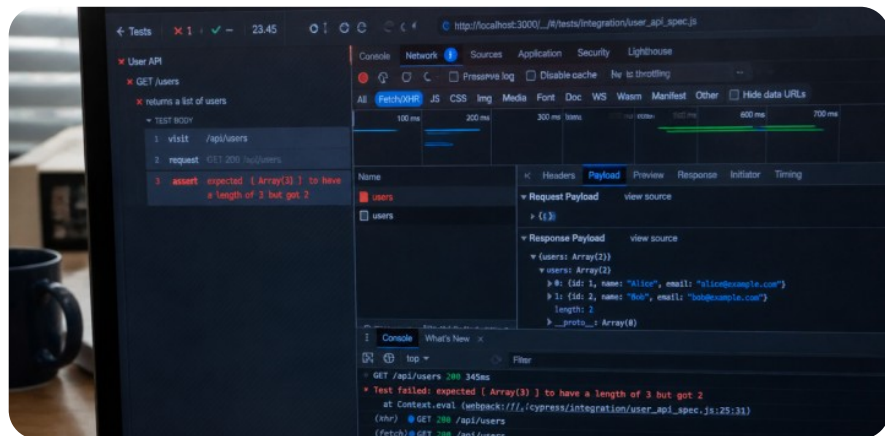
1. Selecionar pela ordem posicional das tags HTML, como `div > p:nth-child(2)`.
2. Utilizar classes CSS de layout e cores, como `.btn-primary-blue`.
3. Utilizar atributos dedicados de teste, como `[data-testid='btn-salvar']`.
4. Copiar o caminho XPath absoluto a partir da raiz `html/body/div`.

Time-Travel Debugging no Cypress



Voltar no Tempo

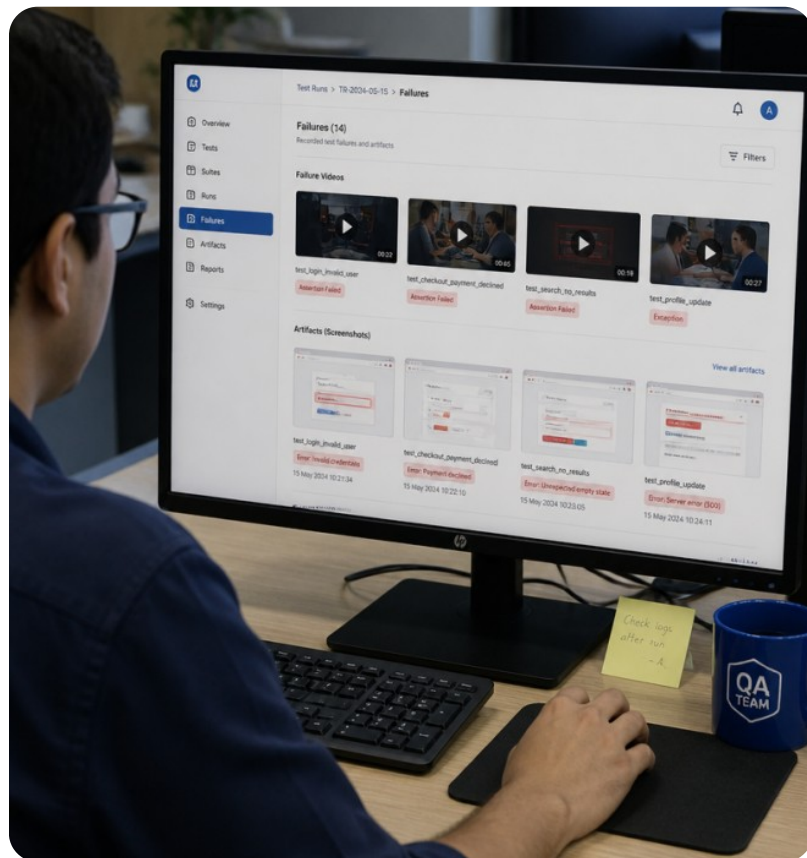
Ao passar o mouse sobre os comandos na lista lateral, o Cypress restaura um **snapshot do DOM** exato daquele milissegundo, permitindo inspecionar cada elemento durante o clique.



Depuração Detalhada

Clicar em um comando imprime dados detalhados no console do DevTools: parâmetros enviados, elementos atingidos e respostas de rede interceptadas.

Screenshots e Gravação de Vídeos



DEBORAÇÃO

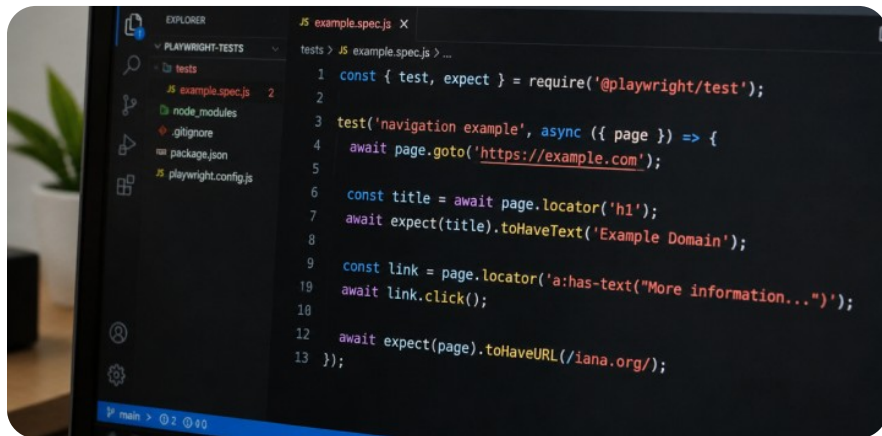
Em ambientes de integração contínua (CI/CD) onde não há monitor físico, as evidências visuais salvam o diagnóstico de falhas:

Screenshots automáticos: capturados no exato momento em que uma asserção falha.

Gravação em vídeo: registro contínuo de toda a execução do navegador em formato MP4.

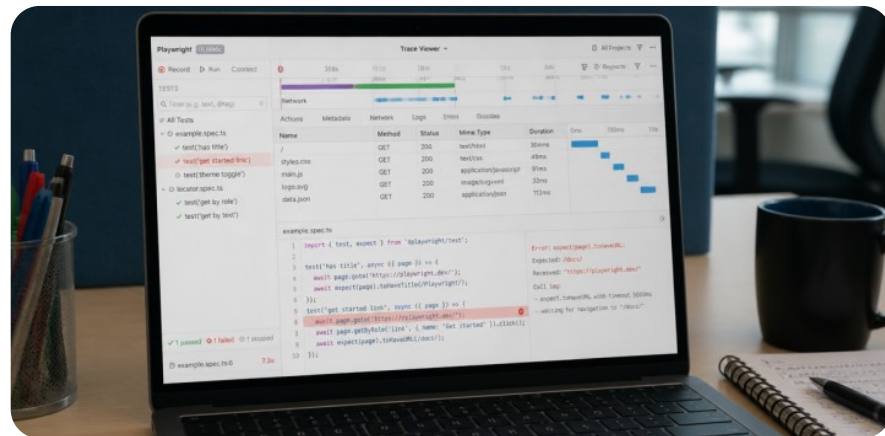
`cy.screenshot('meu-relatorio')`: captura manual sob demanda durante o fluxo.

Automação no Playwright: Sintaxe Async



Async / Await Padrão

No Playwright, cada interação retorna uma Promise nativa:



Auto-Waiting Nativo

O Playwright verifica automaticamente se o botão está visível, habilitado e parado na tela antes de clicar, eliminando pausas manuais instáveis.

Check de Conceito: Pausas Fixas

Inserir pausas forçadas como `cy.wait(5000)` é a melhor solução para testes que falham por lentidão de rede.



VERDADE
IRO



FALSO



Prepare-se para explicar o seu raciocínio.

Check de Conceito: Pausas Fixas

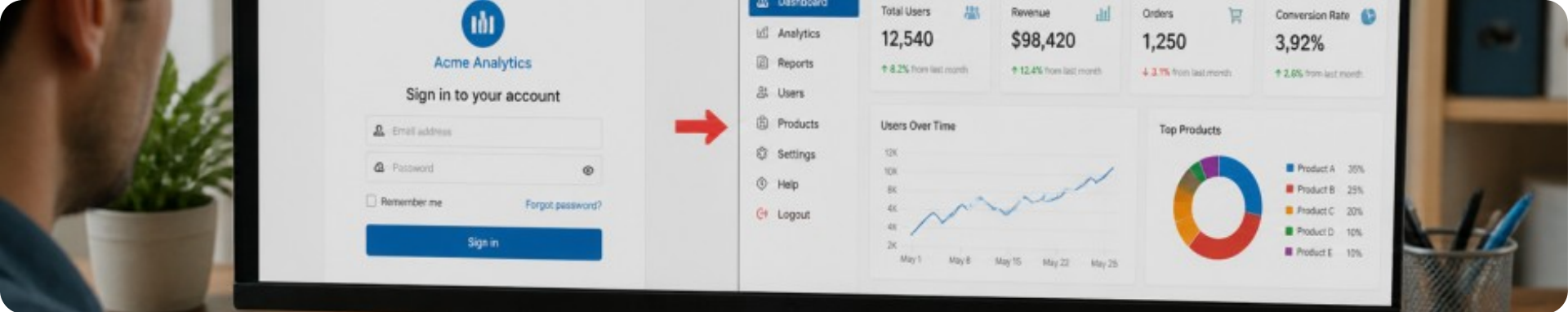


Inserir pausas forçadas como `cy.wait(5000)` é a melhor solução para testes que falham por lentidão de rede.



Por que é isso?

Pausas fixas tornam os testes lentos e não garantem estabilidade; o correto é usar asserções que aguardam estados ou interceptar rotas com `cy.wait('@alias')`.



Roteiro: Fluxo de Autenticação E2E

Vamos analisar a estrutura do primeiro grande desafio prático: o fluxo completo de login e proteção de rotas.

Passo 1: Acessar e validar que o formulário está visível.

- Passo 2: Preencher credenciais válidas e submeter.

Passo 3: Garantir redirecionamento para e saudação ao usuário.

Código do Fluxo de Login (Cypress)

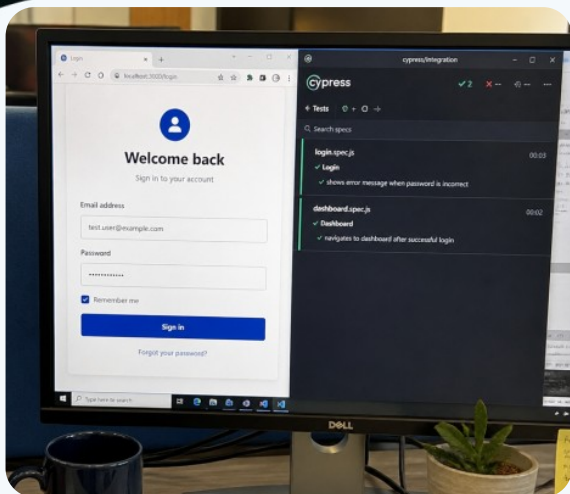
Exemplo de Teste de Autenticação



Ponto-chave

Note que testamos o comportamento real: digitar, clicar e verificar a URL final e o elemento renderizado.

Atividade Prática 1: Login e Dashboard



Implemente testes E2E com Cypress ou Playwright:

1. Sucesso: Acesse `/login`, preencha dados válidos, submeta e valide redirecionamento para `/dashboard` com mensagem de boas-vindas.
1. Falha: Preencha dados inválidos e valide o alerta 'Credenciais inválidas' sem mudar a rota.

A Anatomia de um Fluxo de E-commerce

JORNADA

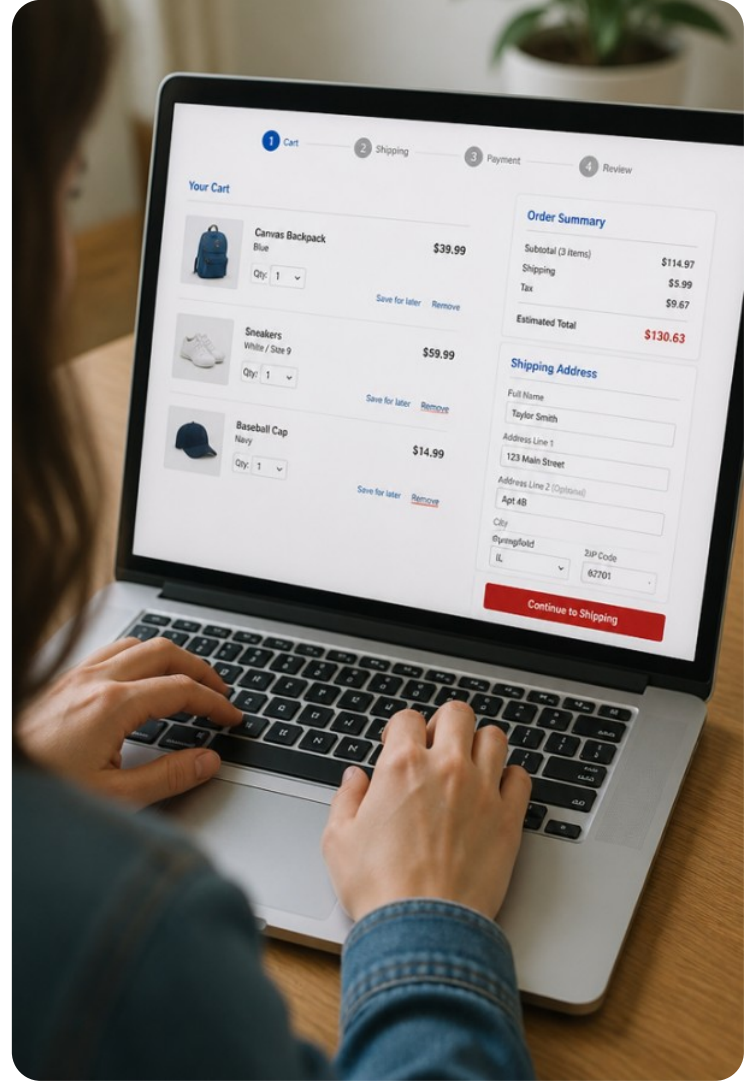
Compras online são o exemplo clássico de teste E2E:

- Catálogo: busca, filtros e seleção.
- Carrinho: adição e conferência do total.
- Checkout: envio e confirmação do pedido.



Lembre-se

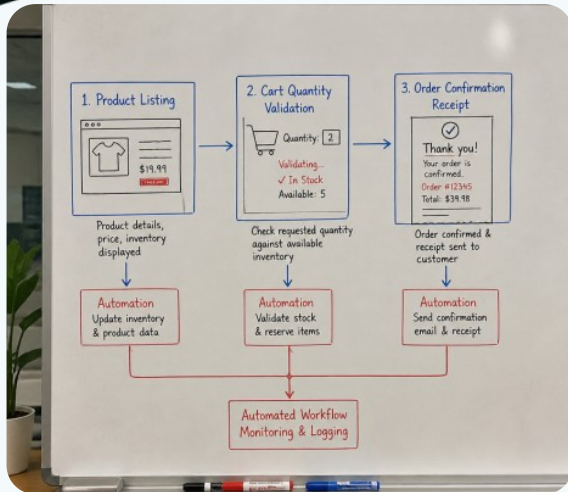
Falhas no fluxo resultam em perda de vendas.



Código do Fluxo de Checkout (Cypress)

Exemplo de Jornada de Compra

Atividade Prática 2: Checkout Completo



Especificação E2E: Jornada de Compra

1.Vitrine: Adicione item ao carrinho.

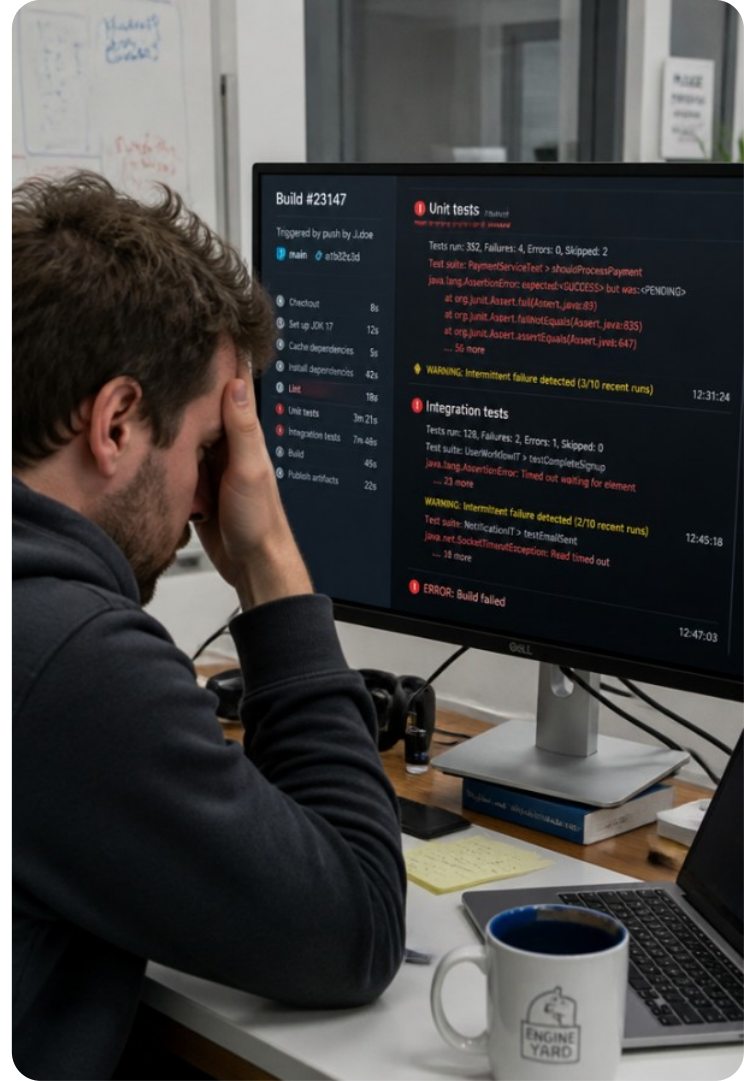
1.Carrinho: Valide quantidade e subtotal.

1.Checkout: Preencha dados e confirme.

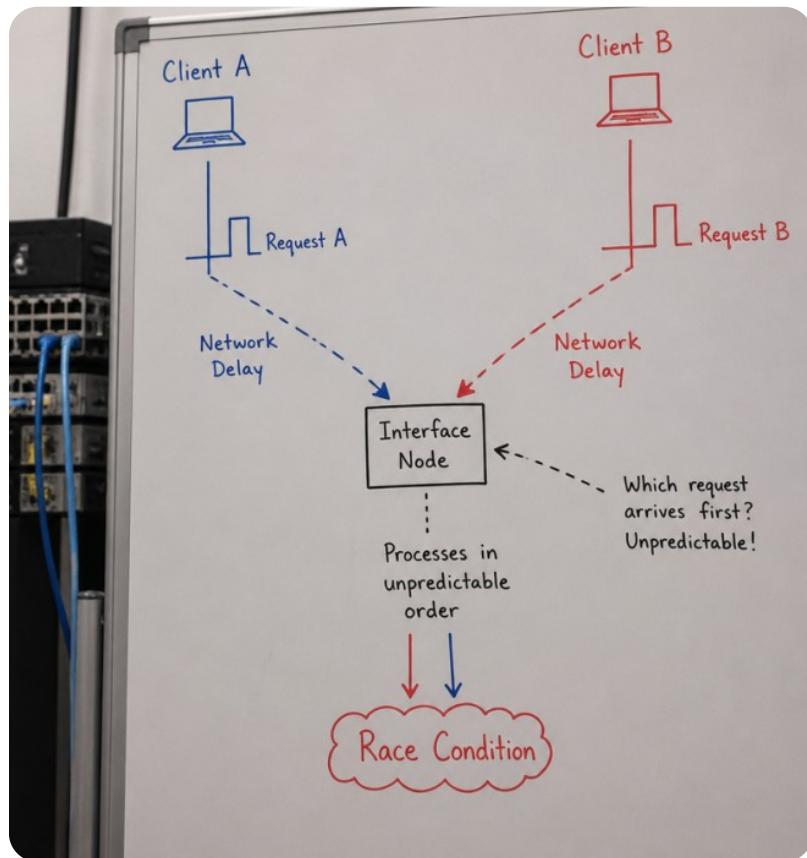
1.Confirmação: Valide mensagem e pedido.

O Pesadelo da Intermitência

O teste passou na sua máquina local, mas falhou duas vezes seguidas na esteira do GitHub Actions sem que ninguém tivesse alterado uma única linha de código. Por que isso acontece?



Causas Comuns de Testes Flaky



ESTABILIDADE

O *flakiness* reduz a confiança nos testes.
Causas comuns:

Race conditions: clique antes da inicialização de eventos.

Dados compartilhados: testes interferem em registros alheios.

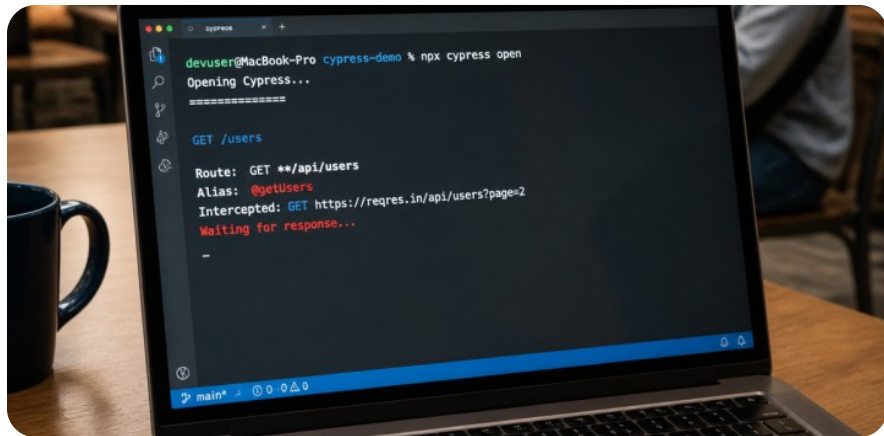
Animações CSS: elementos mudam de posição durante o clique.



Atenção

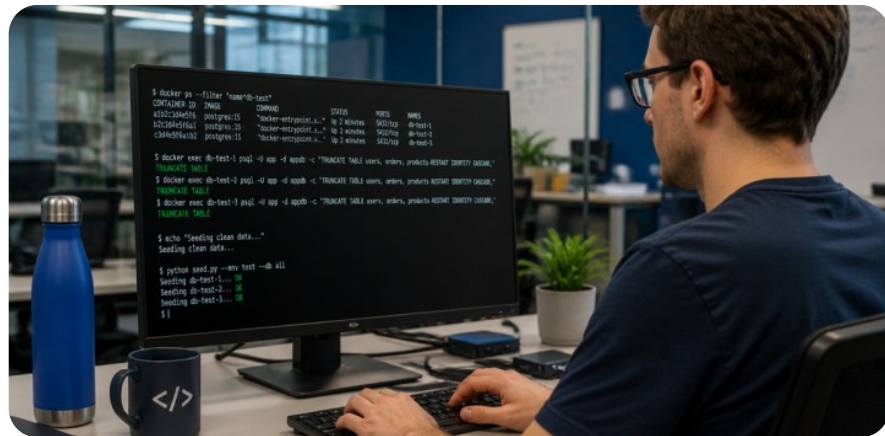
Testes intermitentes ignorados geram ruído e escondem bugs reais.

Técnicas Eficazes de Mitigação



Interceptação Determinística

Use e aguarde explicitamente com `cy.intercept()` em vez de adivinhar o tempo de resposta.



Isolamento de Estado

Garanta que cada teste limpe seus cookies, local storage e banco de teste antes de iniciar, evitando acoplamento entre testes.

Ordenação: Ciclo de Vida de um Teste E2E

Ordene as quatro etapas recomendadas para estruturar a execução segura de um cenário de teste E2E:

Navegar até a rota de entrada da aplicação usando a URL configurada.

Validar os resultados esperados através de asserções encadeadas e verificações de URL.

Preparar o ambiente limpando estado anterior e definindo interceptações de rede.

Executar as ações do usuário simulando entradas de dados e cliques em botões.

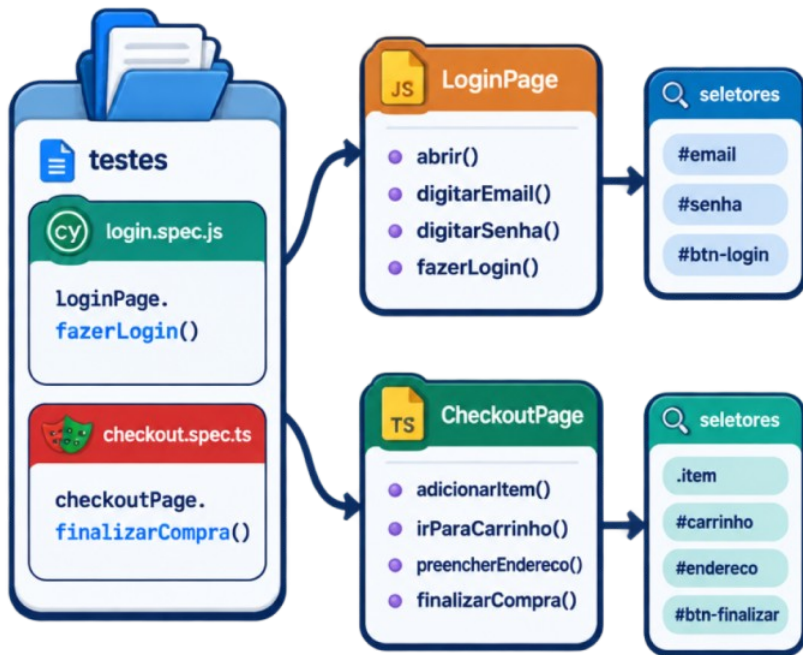
Ordenação: Ciclo de Vida de um Teste E2E

Ordene as quatro etapas recomendadas para estruturar a execução segura de um cenário de teste E2E:



- 1** Preparar o ambiente limpando estado anterior e definindo intercepções de rede.
- 2** Navegar até a rota de entrada da aplicação usando a URL configurada.
- 3** Executar as ações do usuário simulando entradas de dados e cliques em botões.
- 4** Validar os resultados esperados através de asserções encadeadas e verificações de URL.

Page Object Model: Visão Inicial



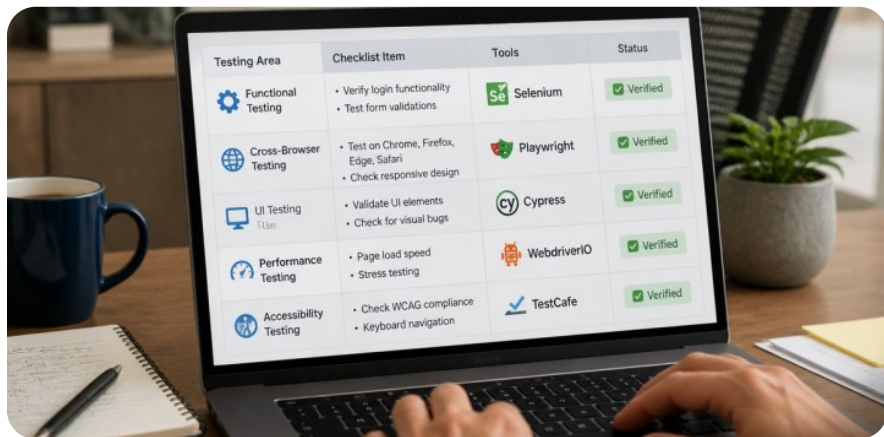
BOAS PRÁTICAS

Conforme a suite cresce, repetir em dezenas de arquivos torna a manutenção inviável:

O padrão **Page Object Model (POM)** encapsula seletores e ações de cada página em classes ou módulos.

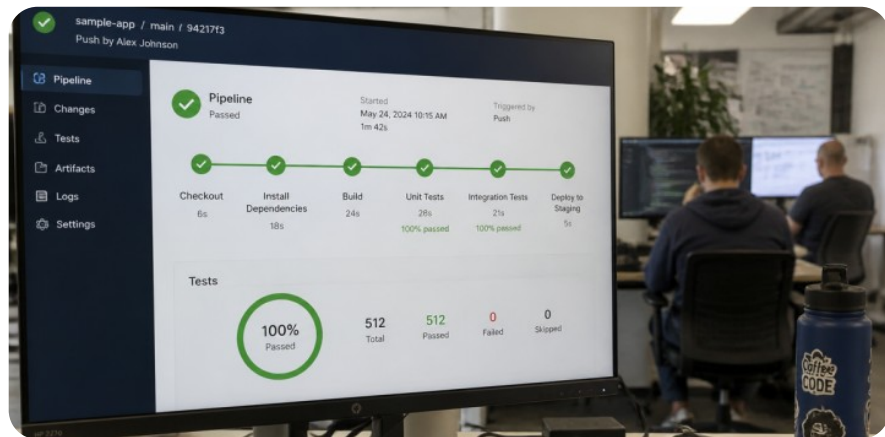
- Quando o HTML muda, você ajusta o seletor em apenas um arquivo centralizado.
- Deixa o código do teste limpo e focado no objetivo do negócio.

Síntese dos Conceitos de E2E



Ferramentas Modernas

Cypress oferece excelente depuração visual com time-travel; Playwright destaca-se pela velocidade, protocolo nativo e automação multi-navegador.



Princípios de Sucesso

Priorize atributos dedicados de teste, controle chamadas de rede com interceptação e mantenha a suite E2E enxuta para validar os fluxos críticos de negócio.

Debate: Cobertura Total em E2E?



Um desenvolvedor propôs: 'Já que os testes E2E testam o navegador real com a maior fidelidade possível, deveríamos substituir todos os nossos testes unitários e de integração por testes E2E'. Quais são os riscos e problemas dessa abordagem para uma equipe de tecnologia?

Debate: Cobertura Total em E2E?



Você poderia ter dito...

Execução lenta: horas em vez de minutos.

Alto custo em CI/CD.

Dificuldade em isolar falhas.

Alta intermitência (flakiness).

Revisão Final: Boas Práticas em E2E

Quais das afirmações a seguir representam boas práticas recomendadas na criação de testes End-to-End? (Selecione as corretas)

1.

Testar todas as variações de validação de formulários exclusivamente na camada E2E.

2.

Aguardar respostas de rede por meio de interceptações em vez de pausas arbitrárias.

3.

Fazer cada teste depender dos dados cadastrados e deixados pelo teste anterior.

4.

Utilizar seletores desacoplados de estilo, como atributos data-testid dedicados.

Revisão Final: Boas Práticas em E2E



Quais das afirmações a seguir representam boas práticas recomendadas na criação de testes End-to-End? (Selecione as corretas)

1.

Testar todas as variações de validação de formulários exclusivamente na camada E2E.

2.

Aguardar respostas de rede por meio de intercepções em vez de pausas arbitrárias.

3.

Fazer cada teste depender dos dados cadastrados e deixados pelo teste anterior.

4.

Utilizar seletores desacoplados de estilo, como atributos data-testid dedicados.